

1/3rd term Programming Fundamentals

Oct. 10 2025, 18:30-21:30h

- You can solve the problems in any order. Solutions must be submitted to the automated judgement system Themis. For each problem, Themis will test ten different inputs, and check whether the outputs are correct.
- Grading: You get one grade point for free. The remaining nine points are based solely on the judgement given by Themis. The first problem is worth one grade point. The remaining four problems are worth two grade points each.
- You can score partial points if a program passes several but not all test cases. For example, if you pass 6 out of 10 test cases, then you are awarded 60% of the grade points for that exercise.
- Inefficient programs may be rejected by Themis. In such cases, the error will be 'time limit exceeded'. The time limit for each problem is one second.
- The number of submissions to Themis is not limited. No points are subtracted for multiple submissions.
- There will be no assessment of programming style. However, accepted solutions are checked manually for cheating: For example, precomputed answers will not be accepted, even though Themis accepts them.
- Needless to say, you are not allowed to work together. If plagiarism is detected, both parties (supplier of the code and the person that sends in copied code) will be excluded from any further participation in the course.
- You are not allowed to use email, phones, smartwatches, tablets, calculators, etc. There is a calculator available on the exam computers (see icon on the desktop). You are allowed to consult the ANSI C book and a dictionary. You are not allowed to use a printed copy of the reader or the lecture slides, however they are available digitally (as a pdf) in Themis. You are allowed to access your own submissions previously made to Themis.
- For each problem, the first three test cases (input files) are available on Themis. These input files, and the corresponding output files, are called `1.in`, `2.in`, `3.in`, `1.out`, `2.out` and `3.out`. These files can be used to test whether the output of your program matches the requested layout, so that there can be no misunderstanding about the layout and spaces in the output.
- **If you fail to pass a problem for a specific test case, then you are advised not to lose much time on debugging your program, and continue with another problem. In the last hour of the exam, all test cases will be made visible in Themis.**

Problem 1: Speeding Fine

Traffic authorities use the ratio between a driver's speed and the legal speed limit to determine fines. The ratio is defined as

$$R = \frac{\text{driver speed}}{\text{speed limit}}.$$

Based on the value of R , the fine category is determined as follows:

Ratio R	Category
$R \leq 1.0$	LEGAL
$1.0 < R \leq 1.2$	MINOR
$1.2 < R \leq 1.5$	MAJOR
$R > 1.5$	SEVERE

Write a program that reads two positive integers: the speed limit (in km/h) and the driver's actual speed (in km/h). The program should print the corresponding fine category.

Example 1:

input:
100 95
output:
LEGAL

Example 2:

input:
100 110
output:
MINOR

Example 3:

input:
80 110
output:
MAJOR

Example 4:

input:
100 170
output:
SEVERE

Problem 2: Max–min replacement until stability

A positive integer n is transformed iteratively as follows. Let $\max$ be the largest digit of n and let $\min$ be the smallest digit of n . Compute $d = \max - \min$ and replace *every occurrence of the digit* $\max$ in n by d (all other digits remain unchanged). Remove any leading zeros that may appear.

The number n is considered *stable* if applying the transformation does not change it anymore. The process stops when stability is reached.

For example, beginning with **582** the process evolves as follows:

582 $\longrightarrow$ 562 $\longrightarrow$ 542 $\longrightarrow$ 342 $\longrightarrow$ 322 $\longrightarrow$ 122 $\longrightarrow$ 111 $\longrightarrow$ 0.

The final stable number is 0, reached after 7 transformation steps.

The input is a positive integer n . The program should output two numbers, separated by a space: the final stable number obtained after repeatedly applying the transformation, and the number of steps required to reach it.

Example 1:

input:
582
output:
0 7

Example 2:

input:
909
output:
909 0

Example 3:

input:
314
output:
0 4

Problem 3: Remainder squash length

Define a sequence $(a_k)_{k \geq 0}$ by

$$a_0 = n, \quad a_k = \sum_{d \in \text{digits}(a_{k-1})} (d^2 \bmod m) \quad \text{for } k \geq 1,$$

where $\text{digits}(x)$ are the decimal digits of x . Because the values $d^2 \bmod m$ are bounded, the sequence eventually repeats. The repeating part is called the *squash cycle*, and its length is the number of distinct terms in that cycle.

For example, for $n = 7$ and $m = 35$ we find the cycle length 5 (cycle is $[15, 26, 5, 25, 29]$):

$$7 \longrightarrow 14 \longrightarrow 17 \longrightarrow 15 \longrightarrow 26 \longrightarrow 5 \longrightarrow 25 \longrightarrow 29 \longrightarrow 15 \longrightarrow \dots$$

The input for this problem are two integers n and m , where $0 \leq n < 10^9$ and $1 \leq m \leq 1000$. The output must be the length of the squash cycle in the sequence that starts with $a_0 = n$.

Example 1:

input:

7 35

output:

5

Example 2:

input:

2 22

output:

3

Example 3:

input:

57 10

output:

1

Problem 4: Pair cancellation

We define a reduction on a lowercase word as follows: while the word contains two equal neighbouring letters, delete that adjacent pair. Repeat this process until no such pair remains. If the word becomes empty, we say its reduced form is the empty word.

Input. A single word of at most 20 lowercase English letters.

Output. The reduced word after repeatedly deleting adjacent equal-letter pairs. If the reduced word is empty, output `###`.

Example 1:

input:

velle

output:

v

Example 2:

input:

aabccb

output:

###

Example 3:

input:

abcd

output:

abcd

Problem 5: Digit inventory numbers

You are given a starting integer n with $0 \leq n < 100$. We define an iterative process on the number as follows: Read the decimal digits of the current number and count how many times each digit $d \in \{0, 1, \dots, 9\}$ occurs. Form the next number by writing, in increasing digit order, the *count* (in decimal, without leading zeros) followed by the *digit* d , but only for those digits that occur at least once. Concatenate these pairs to obtain the next number.

Example. Starting from $n = 42$:

$$42 \rightarrow 1214 \rightarrow 211214 \rightarrow 312214 \rightarrow \dots$$

Explanation:

- 42: one 2, one 4 $\Rightarrow$ 12 14 $\Rightarrow$ 1214.
- 1214: two 1s, one 2, one 4 $\Rightarrow$ 21 12 14 $\Rightarrow$ 211214.
- 211214: three 1s, two 2s, one 4 $\Rightarrow$ 31 22 14 $\Rightarrow$ 312214.

The input for this problem consists of two numbers n (where $0 \leq n < 100$) and s (where $0 \leq s \leq 20$). The output must be the number that is obtained by applying the above transformation exactly s times, starting from the decimal representation of n . Numbers are written without spaces.

Example 1:

input:

42 3

output:

312214

Example 2:

input:

42 1

output:

1214

Example 3:

input:

0 5

output:

10311213